Cache Oblivious Algorithm Design

- Argues that it is possible to, at least in theory, design algorithms that do not need any tuning.
- Remember the basic model of execution:
  - Computer can reference words from its nearest level of memory, the cache.
  - If the referenced word exists in the cache, a cache hit occurs.
  - Otherwise, a cache miss occurs. In this case, the required word, plus a contiguous portion equaling the cache line size, is brought into the cache from the next level of memory.

Cache Oblivious Algorithm Design

- An ideal cache model:
  - Two level memory model
  - M – size of the faster memory, and
  - B – size of the transfer into the faster memory from the smaller memory.
  - Cache can evict a block of size B if there is no place for the incoming block.
  - Four assumptions follow.

Cache Oblivious Algorithm Design

1. Optimal Replacement : The model assumes an optimal replacement policy (FIF).

Cache Oblivious Algorithm Design

1. Optimal Replacement : The model assumes an optimal replacement policy (FIF).
2. Two Levels of Memory with the Inclusion property: Data cannot be in level 1 unless it is present at level 2.

Cache Oblivious Algorithm Design

1. Optimal Replacement : The model assumes an optimal replacement policy (FIF).
2. Two Levels of Memory with the Inclusion property: Data cannot be in level 1 unless it is present at level 2.
3. Full Associativity : What does this mean?

Cache Oblivious Algorithm Design

1. Optimal Replacement : The model assumes an optimal replacement policy (FIF).
2. Two Levels of Memory with the Inclusion property: Data cannot be in level 1 unless it is present at level 2.
3. Full Associativity : A block of data fetched from Level 2 can reside anywhere in Level 1.

Cache Oblivious Algorithm Design

1. Optimal Replacement : The model assumes an optimal replacement policy (FIF).
2. Two Levels of Memory with the Inclusion property: Data cannot be in level 1 unless it is present at level 2.
3. Full Associativity : A block of data fetched from Level 2 can reside anywhere in Level 1.
4. Automatic Replacement: The algorithm designer need not call for cache replacement.

Model

- An algorithm that causes Q(n;M;B) cache misses on a problem of size n using an ideal cache incurs $Q(n;M;B) \leq 2Q^*(n; M/2 ;B)$ cache misses on a (M;B) cache that uses LRU, FIFO replacement.
- This is only true for algorithms which follow a regularity condition.
- Regularity Condition: An algorithm satisfies the regularity condition if it slows down by a constant factor when the amount of memory (M) is reduced by half.

Basic Tools

- Scan of an array : Takes at most O(ceil(N/B)) I/O operations.
- This is without knowing B. But, data is always moved in units of B.

Basic Tools

- Divide and Conquer: A popular algorithm design mechanism too.
- A good mechanism for parallel algorithm design also.
- Allows for a simple and elegant analysis for the number of I/O operations.
- Turns out that most such algorithms are indeed cache oblivious.
  - Some are optimal too in this model.

Basic Tools

- Why Divide-and-Conquer?
  - The philosophy is tuned to the model as follows.
  - Divide the problem until a subproblem fits into a cache block of size B.
  - At this point, no further cache misses occur.
  - Hence, the required analysis can be modeled as a recurrence relation.
  - The recurrence relation includes the number of cache misses incurred during the divide phase.

Example: Quick Sort

- Choosing a pivot uniformly at random has at most one cache miss.
- Splitting the input around the chosen pivot is identical to a scan in terms of cache misses.
- This proceeds recursively.
- Once the problem size reaches B, or smaller than B, then no further I/O transer.

Example: Quick Sort

- Average number of I/Os guided by the following relation.

$$Q(N) = \frac{1}{N}\left[\sum_{i=1,\ldots,\frac{N-1}{B}} \left(Q(i)+Q(N-i)\right) + O\left(1+\frac{N}{B}\right)\right]$$

Example: Quick Sort

- Average number of I/Os guided by the following relation.

$$Q(N) = \frac{1}{N}\left[\sum_{i=1,\ldots,\frac{N-1}{B}} \left(Q(i)+Q(N-i)\right) + O\left(1+\frac{N}{B}\right)\right]$$

- The solution to the above relation is

$$Q(N) = O\left(\frac{N}{B}\log_2 \frac{N}{B}\right)$$

Finding the Median

- Recall the linear time median algorithm.
- Take it as a Reading Exercise if you are not aware of this algorithm.
- This algorithm groups elements into groups of 5, computes the median of each group, the median-of-the-medians recursively, splits the input accordingly, and recurses on a subproblem of size at most 7N/10 + 6.

Finding the Median

- The recurrence relation for the number of I/O operations is easily seen to be:

$$Q(N) = Q\left(N/5\right) + Q(7N/10 + 6) + O(N/B)$$

Finding the Median

- The recurrence relation for the number of I/O operations is easily seen to be:

$$Q(N) = Q\left(N/5\right) + Q(7N/10 + 6) + O(N/B)$$

- The solution for the above recurrence is

$$Q(N) = O(N/B)$$

Matrix Multiplication

- Recall Strassen's algorithm.
- Multiplying two nxn matrices is done as 7 matrix multiplications on n/2xn/2 matrices, plus some n/2xn/2 matrix additions.
- The recurrence relation for the time taken is

$$T(n) = 7T(n/2) + O(n^2)$$

Matrix Multiplication

- Recall Strassen's algorithm.
- Multiplying two n$\times$n matrices is done as 7 matrix multiplications on n/2$\times$n/2 matrices, plus some n/2$\times$n/2 matrix additions.
- The recurrence relation for the time taken is

$$T(n) = 7T(n/2) + O(n^2)$$

which solves to $T(n) = O(n^{\log_2 7})$.

Matrix Multiplication

- For Strassen's algorithm, the number of I/O operations is given by the following recurrence.

$$Q(N) \leq \begin{cases} O(\frac{N^2}{B}) \; if \quad N^2 \leq \alpha M \\ 7Q(N/2) + O(\frac{N^2}{B}) \quad otherwise \end{cases}$$

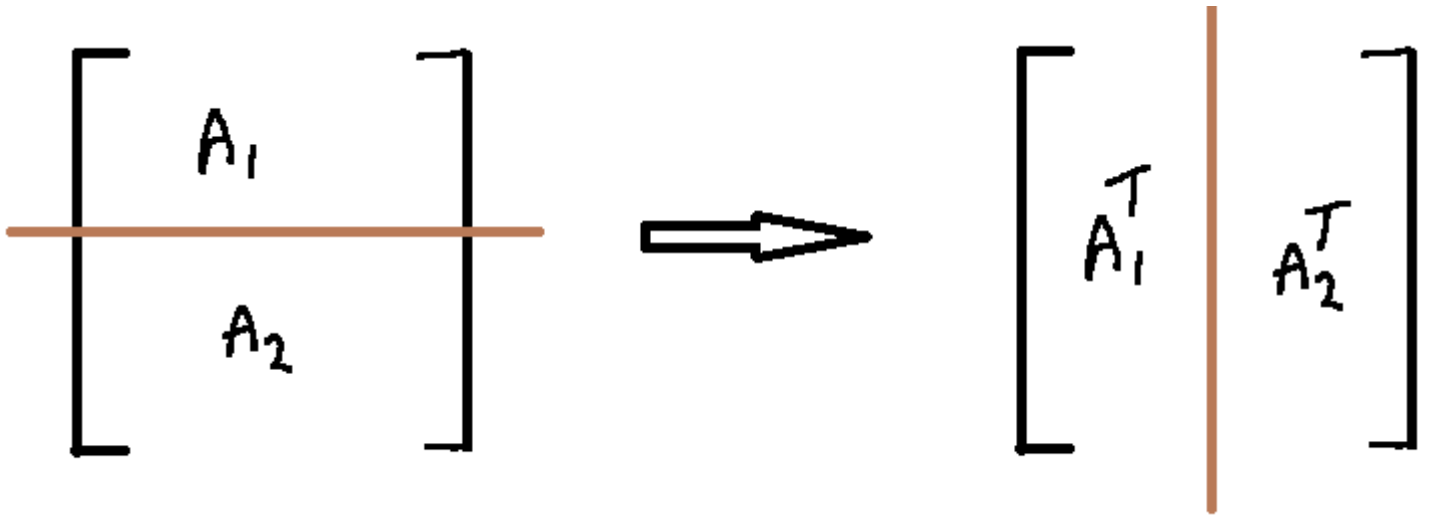
- The term $O(N^2/B)$ is for adding the matrices.
- The solution for the above is $Q(N) = O\left(\frac{N^2}{B} + \frac{N^{\log_2 7}}{B\sqrt{M}}\right)$.

Matrix Transpose

- Quite challenging for a simple problem.
- Trouble is that matrices are normally stored in a row-major form.
- Transpose requires writing in column-major form.
- Exercise: What is the number of cache misses suffered by the following standard approach?

```
for i = 1 to n do
    for j = 1 to n do
        B[j][i]  = A[i][j]
    endfor
endfor
```

Matrix Transpose



- Recursively break the matrix into two matrices, splitting the larger dimension.
- If the size of the matrix is in O(B), then the transpose is done without further splitting.
- On a nxn matrix, one can show that the number of cache misses is $O(N^2/B)$.
- Can carefully extend the analysis to an nxp matrix.

Matrix Transpose

- The recurrence relation for this process is

$Q(n,p) = 2Q(n/2, p) + O(1)$ assuming $n > p$

- Depending on the progress of the algorithm in each recursive call, the recurrence relation changes slightly.

- The base case is when the size of the matrix for the recursive call is within O(B).
- Cache misses at that point equals that of scan().

- All these solve to $Q(n,p) = O(np/B)$.

Several Others…

- See for more algorithms designed in the cache oblivious model for
  - Sorting
  - FFT
  - Data structures and operations on data structures for large data,
  - …

Still, Lot Left to Be Done.

- How to integrate prefetching into the cache aware or cache oblivious model?
- How do these models work in a multi-core setting?
- Study the practicality of these algorithms. Is asymptotics misleading? Some recent research suggests so in this case.
- Several problems for which optimal cache oblivious algorithms are not known
  - Sorting variable length data such as strings
  - Shortest paths
  - Convex hull

Further Reading

- The original paper that introduced cache oblivious algorithms, written by Leiserson et al., FOCS 2002
- An experimental evaluation, written by Pingali et al., SPAA 2007.
- A survey notes by Piyush Kumar, available on the course website.
- A survey by Vitter on Cache Aware Algorithms in ACM Computing Surveys.